React Custom Hooks: Production Patterns & Best Practices

MERN Stack Bootcamp Lecture
Instructor: Kashif Raza | LinkedIn
Duration: ~60 minutes

🎯 Lecture Objectives

1. Extract component logic into **reusable, testable custom hooks**
2. Follow industry conventions and React's strict hook rules
3. Implement 4 production-grade hooks used in real-world applications
4. Compose multiple hooks cleanly for complex features
5. Avoid common anti-patterns that cause memory leaks or stale state

> 💡 **Mindset Shift:**
> `Components = UI`
> `Custom Hooks = Business Logic`
> If it doesn't render JSX, it probably belongs in a hook.

📏 The 3 Non-Negotiable Rules

Rule	Why It Matters
Start with `use`	ESLint & React rely on this to detect hook usage
Call at the top level	React tracks state by call order; conditionals break it
Only call from React functions	Components or other custom hooks only

```
# Enable enforcement
npm install eslint-plugin-react-hooks --save-dev
```

```
// .eslintrc.js
rules: {
  'react-hooks/rules-of-hooks': 'error',
  'react-hooks/exhaustive-deps': 'warn'
}
```

🏭 1. `useLocalStorage` → Persistent State

Use Case: User preferences, theme, cart, form drafts. Used in Vercel dashboard, GitHub settings, e-commerce checkouts.

```
// hooks/useLocalStorage.js
import { useState, useEffect } from 'react';

/**
 * Sync React state with localStorage safely
 * @param {string} key
 * @param {any} initialValue
 * @returns {[any, Function]}
 */
export function useLocalStorage(key, initialValue) {
  const [storedValue, setStoredValue] = useState(() => {
    if (typeof window === 'undefined') return initialValue;
    try {
      const item = localStorage.getItem(key);
      return item ? JSON.parse(item) : initialValue;
    } catch {
      return initialValue;
    }
  });

  useEffect(() => {
    try {
      if (storedValue === undefined) {
        localStorage.removeItem(key);
      } else {
        localStorage.setItem(key, JSON.stringify(storedValue));
      }
    } catch (error) {
      console.warn(`localStorage error for "${key}":`, error);
    }
  }, [key, storedValue]);

  const setValue = (value) => {
    try {
      const valueToStore = value instanceof Function ? value(storedValue) : value;
      setStoredValue(valueToStore);
    } catch (error) {
      console.warn(`Error setting "${key}":`, error);
    }
  };

  return [storedValue, setValue];
}
```

Usage:

```
import { useLocalStorage } from './hooks/useLocalStorage';

function ThemeToggle() {
  const [theme, setTheme] = useLocalStorage('app-theme', 'light');

  return (
    <button onClick={() => setTheme(t => t === 'light' ? 'dark' : 'light')}>
      Current: {theme}
    </button>
```

```
  );
}
```

🏭 2. `useDebounce` → Input/Scroll Optimization

Use Case: Search bars, API autosave, scroll listeners. Used in Stripe, Algolia, Notion.

```
// hooks/useDebounce.js
import { useState, useEffect } from 'react';

export function useDebounce(value, delay = 300) {
  const [debouncedValue, setDebouncedValue] = useState(value);

  useEffect(() => {
    const timer = setTimeout(() => setDebouncedValue(value), delay);
    return () => clearTimeout(timer); // Cleanup prevents stale updates
  }, [value, delay]);

  return debouncedValue;
}
```

Usage:

```
import { useState } from 'react';
import { useDebounce } from './hooks/useDebounce';
import { useFetch } from './hooks/useFetch';

function SearchBar() {
  const [query, setQuery] = useState('');
  const debouncedQuery = useDebounce(query, 400);
  const { data: results, loading } = useFetch(
    debouncedQuery ? `/api/search?q=${debouncedQuery}` : null
  );

  return (
    <div>
      <input value={query} onChange={e => setQuery(e.target.value)} />
      {loading && <span>Searching...</span>}
      <ResultsList items={results} />
    </div>
  );
}
```

🏭 3. `useFetch` → Safe API Calls

Use Case: Data fetching with loading, error, retry, and abort handling. Used in dashboards, feeds, admin panels.

```
// hooks/useFetch.js
import { useState, useEffect, useCallback } from 'react';

export function useFetch(url, options = {}) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  const fetchData = useCallback(async () => {
    if (!url) { setLoading(false); return; }

    const controller = new AbortController();
    setLoading(true);
    setError(null);

    try {
      const res = await fetch(url, { ...options, signal: controller.signal });
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      setData(await res.json());
    } catch (err) {
      if (err.name !== 'AbortError') setError(err.message);
    } finally {
      setLoading(false);
    }

    return () => controller.abort();
  }, [url, JSON.stringify(options)]);

  useEffect(() => {
    const cleanup = fetchData();
    return cleanup;
  }, [fetchData]);

  return { data, loading, error, refetch: fetchData };
}
```

Usage:

```
const { data: user, loading, error, refetch } = useFetch('/api/users/me');
if (loading) return <Spinner />;
if (error) return <Alert type="error" message={error} />;
return <Profile user={user} />;
```

🏭 4. `useClickOutside` → Dropdown/Modal UX

Use Case: Close modals, menus, popovers on outside click. Used in Tailwind UI, shadcn/ui, MUI.

```
// hooks/useClickOutside.js
import { useEffect, useRef } from 'react';

export function useClickOutside(handler, listenCapturing = true) {
  const ref = useRef(null);

  useEffect(() => {
    const listener = (e) => {
      if (!ref.current || ref.current.contains(e.target)) return;
      handler(e);
    };

    document.addEventListener('mousedown', listener, listenCapturing);
    return () => document.removeEventListener('mousedown', listener, listenCapturing);
  }, [handler, listenCapturing]);

  return ref;
}
```

Usage:

```
import { useState } from 'react';
import { useClickOutside } from './hooks/useClickOutside';

function Dropdown() {
  const [open, setOpen] = useState(false);
  const dropdownRef = useClickOutside(() => setOpen(false));

  return (
    <div ref={dropdownRef} className="relative">
      <button onClick={() => setOpen(o => !o)}>Menu ▼</button>
      {open && (
        <div className="absolute top-full left-0 bg-white shadow-lg p-4">
          <a href="#link1">Option 1</a>
          <a href="#link2">Option 2</a>
        </div>
      )}
    </div>
  );
}
```

🔗 Hook Composition: Real-World Pattern

Industry apps **chain hooks** instead of writing monolithic components.

```
// hooks/useSearch.js (Composition Example)
import { useState } from 'react';
import { useDebounce } from './useDebounce';
import { useFetch } from './useFetch';

export function useSearch(apiEndpoint, delay = 300) {
  const [query, setQuery] = useState('');
  const debounced = useDebounce(query, delay);
  const { data: results, loading, error, refetch } = useFetch(
    debounced ? `${apiEndpoint}?q=${debounced}` : null
  );

  return {
    query,
    setQuery,
    results,
    loading,
    error,
    clear: () => setQuery('')
  };
}

// Usage in 10 lines:
function ProductSearch() {
  const { query, setQuery, results, loading, error } = useSearch('/api/products');
  return (
    <div>
      <input value={query} onChange={e => setQuery(e.target.value)} />
      {loading && <p>Loading...</p>}
      {error && <p className="text-red-500">{error}</p>}
      <ProductGrid items={results} />
    </div>
  );
}
```

🧪 Testing & Debugging Custom Hooks

Quick Testing Strategy

You don't need heavy setup. Wrap the hook in a minimal component.

```jsx
// __tests__/useLocalStorage.test.jsx
import { render, screen, fireEvent } from '@testing-library/react';
import { useLocalStorage } from '../hooks/useLocalStorage';

function TestWrapper({ hookFn, children }) {
  const result = hookFn();
  return children(result);
}

test('persists and updates value', () => {
  render(
    <TestWrapper hookFn={() => useLocalStorage('test', 'init')}>
      {([val, set]) => (
        <div>
          <span data-testid="value">{val}</span>
          <button onClick={() => set('updated')}>Update</button>
        </div>
      )}
    </TestWrapper>
  );

  expect(screen.getByTestId('value')).toHaveTextContent('init');
  fireEvent.click(screen.getByText('Update'));
  expect(screen.getByTestId('value')).toHaveTextContent('updated');
  expect(localStorage.getItem('test')).toBe('"updated"');
});
```

Debugging Tips

- `useDebugValue(val)` → Shows custom hook state in React DevTools
- `console.log` inside hooks runs on every render → use sparingly
- React DevTools → **Components** tab → expand **Hooks** section to inspect live state

✅ Best Practices Checklist

Do	Don't
Start every hook with `use`	Call hooks inside loops/conditions
Return objects or arrays consistently	Mix return types across versions
Handle unmount & cleanup	Leave intervals/listeners attached
Document params/returns with JSDoc	Hide side effects from consumers
Keep hooks single-purpose	Build "god hooks" that do 5 things
Use `useCallback` for returned functions	Return new functions on every render

🚫 When NOT to Use Custom Hooks

- Simple one-off state → use `useState` inline
- Pure UI logic → keep it in the component
- Heavy state management → use Zustand/Redux instead
- You're duplicating a well-tested library hook → just install it

🧪 Mini Lab: Build `useMediaQuery` (15 mins)

Goal: Detect screen breakpoints for responsive components without CSS-in-JS bloat.

Starter:

```
// hooks/useMediaQuery.js
import { useState, useEffect } from 'react';

export function useMediaQuery(query) {
  const [matches, setMatches] = useState(() => {
    if (typeof window === 'undefined') return false;
    return window.matchMedia(query).matches;
  });

  useEffect(() => {
    const mql = window.matchMedia(query);
    const handler = (e) => setMatches(e.matches);

    mql.addEventListener('change', handler);
    return () => mql.removeEventListener('change', handler);
  }, [query]);

  return matches;
}
```

Task:

1. Implement the hook above
2. Use it in a component to conditionally render a mobile menu vs desktop nav
3. Test resizing the browser window → verify instant UI switch without CSS media queries

✅ **Success Criteria:** Works in SSR-safe way, cleans up listeners, returns boolean.

📦 Recommended Production Libraries

Need	Library	Why
Data Fetching	`@tanstack/react-query` or `swr`	Caching, retries, background updates
Forms	`react-hook-form`	Performant, uncontrolled, hooks-native
State	`zustand` or `jotai`	Lightweight, hook-based, no boilerplate
UI Primitives	`@radix-ui/react-*`	Accessible, headless, hook-driven

> 💡 **Rule:** Don't reinvent. Build custom hooks for **your business logic**, not generic utilities that already exist in battle-tested packages.

🎯 Instructor Closing

Custom hooks are the **glue** between clean UI and scalable architecture. When used correctly:

- Components shrink to 30–50 lines
- Logic becomes unit-testable
- Teams share patterns instead of copying code
- Bugs drop because side effects are centralized

Your next step: Pick one repeated `useEffect` / `useState` block from your current project. Extract it. Name it `use______`. Test it. Import it. Repeat.

I'll be reviewing your lab submissions and hopping on calls for architecture reviews. Keep your hooks pure, your cleanup tight, and your dependencies explicit.

— Kashif Raza | Composable logic, production-ready UI 🪝

Document Version: 1.0 | Last Updated: $(date)